

A COOKBOOK FOR ENGINEERS

Deep Learning Cookbook

Neural networks in PyTorch, built from the tensor up — training loops, CNNs, transfer learning, and how to ship a model — for a developer who knows Python and some ML.

PyTorch

Tensors

Autograd

nn.Module

Training loops

CNNs

Transfer learning

GPU

Written for a developer who knows Python and classic ML, and now wants neural networks for real — beyond the 'taste' in the ML book.

Every chapter answers: what it is, how to build it in PyTorch, the pitfalls, and when a simpler model wins.

Edition 2026 · intuition + working code · models that train and ship

Table of Contents

How to Use This Book	4
The one idea: layers of learned transformations, trained by gradients	4
What this book covers	5
Part 1 · When Deep Learning (and When Not)	6
1.1 What deep learning is good at	6
1.2 What deep learning is not the answer for	6
1.3 The data and compute reality	7
Part 2 · Tensors — the Data Structure	8
2.1 Creating and inspecting	8
2.2 Shapes are the whole game	8
2.3 Operations, broadcasting, and reshaping	9
2.4 GPU tensors	9
Part 3 · Autograd — How Learning Actually Happens	10
3.1 Tensors that track history	10
3.2 The four-step dance (the heart of every training loop)	10
3.3 Why <code>zero_grad()</code> matters	11
3.4 Turning autograd off for inference	11
Part 4 · Building Networks with <code>nn.Module</code>	12
4.1 The layers you'll use constantly	12
4.2 Defining a model	12
4.3 Loss functions and what the last layer outputs	13
Part 5 · The Training Loop	14
5.1 The canonical loop	14
5.2 Optimizers and learning rate	14
5.3 Always validate on held-out data	15
5.4 Read the two loss curves together	15
Part 6 · Feeding Data: <code>Dataset</code> & <code>DataLoader</code>	16
6.1 <code>Dataset</code> — one example at a time	16
6.2 <code>DataLoader</code> — batching, shuffling, parallelism	16
6.3 Why batches at all	17
6.4 Normalize your inputs	17
Part 7 · Convolutional Networks for Images	18
7.1 Why not just a big Linear layer?	18
7.2 The building blocks	18
7.3 Getting the flattened size right	19
7.4 In practice, you won't build from scratch	19
Part 8 · Overfitting & Regularization	20
8.1 Spotting overfitting	20
8.2 The regularization toolkit	20
8.3 eval vs train mode really matters here	21

Part 9 · Transfer Learning — the Technique You'll Actually Use	22
9.1 Why it works.	22
9.2 The recipe (image example).	22
9.3 It's not just images	23
Part 10 · GPUs, Saving & Serving	24
10.1 GPU training, portably	24
10.2 Saving and loading — save the state_dict	24
10.3 Serving behind an API	25
10.4 Beyond a plain endpoint	25
Part 11 · Capstone & Cheat Sheet	27
11.1 Capstone — train and ship an image classifier.	27
11.2 PyTorch cheat sheet	27
11.3 Debugging cheat sheet	27
11.4 The five rules.	28

How to Use This Book

Your Machine Learning cookbook gave deep learning "a taste." This book is the full meal: **neural networks in PyTorch**, built from the tensor up, until you can train a real model on your own data and ship it. It assumes you know Python and the ML basics (train/test splits, overfitting, loss) and want the piece that powers modern vision, language, and audio.

Deep learning isn't magic and it isn't always the answer — for tabular data, gradient boosting usually still wins (we'll say so plainly). But when the input is *perceptual* — images, text, sound — or the patterns are too complex to hand-engineer features for, neural networks are the tool, and PyTorch is how most of the world builds them.

Each chapter answers four questions:

1. **What is this and when do you use it?**
2. **How do you build it in PyTorch?**
3. **What are the pitfalls?**
4. **When is a simpler model the better call?**

The one idea: layers of learned transformations, trained by gradients

A neural network is a stack of simple transformations (**layers**), each with tunable **weights**, that together turn an input into an output. You define a **loss** (how wrong it is), and an optimizer nudges every weight a little in the direction that reduces the loss — **gradient descent** — over and over.

DIAGRAM

```

input → [layer] → [layer] → [layer] → output → loss (vs truth)
                                     |
                                     ↓
          adjust every weight to reduce loss ← (backprop, many times)
  
```

That loop — forward pass, measure loss, backpropagate gradients, update weights — is the entire engine. Everything else is architecture (what the layers are) and logistics (data, regularization, hardware).

ENGINEER'S MENTAL MODEL

A model is a function with millions of tunable constants, and *training* is an optimization loop that sets those constants by trial and gradient. You don't write the logic; you define the shape (the layers) and the objective (the loss), and the optimizer discovers the weights. Your job is architecture, data quality, and the training recipe — not hand-coding the mapping.

What this book covers

The core building blocks — **tensors** and **autograd** — then **nn.Module** and the **training loop**, efficient **data loading**, **CNNs** for images, **regularization** to stop overfitting, **transfer learning** (the technique you'll actually use most), and **shipping**: GPU, saving, and serving. It favours working PyTorch you can run over derivations.

HOW TO READ IT

Read Parts 1-5 strictly in order — tensors → autograd → modules → training loop are a single idea assembled step by step, and skipping breaks it. Parts 6-9 are the practical toolkit. Have PyTorch installed and run every snippet; deep learning is muscle memory. A free GPU (Colab/Kaggle) helps from Part 7 on, but everything here runs on CPU for learning.

READ THIS FIRST: YOU MAY NOT NEED DEEP LEARNING

For **tabular data** (rows and columns, the most common business case), gradient-boosted trees (XGBoost/LightGBM, in the ML book) usually **beat** neural nets while training in seconds on a laptop and needing far less data and tuning. Reach for deep learning when the input is perceptual (images, text, audio) or the dataset is large and the patterns defeat feature engineering. If your data is a spreadsheet, start with boosting.

Part 1 · When Deep Learning (and When Not)

Deep learning is powerful and often oversold. Before the mechanics, the judgement: what problems it's genuinely the right tool for, and the (large) set where a simpler model is faster, cheaper, and better. Getting this call right saves weeks.

1.1 What deep learning is good at

Neural networks excel when the input is **high-dimensional and perceptual**, and when you have **enough data** for them to learn features you'd struggle to engineer by hand:

Domain	Example	Why DL wins
Images	classify, detect, segment	learns visual features from pixels
Text / language	translation, LLMs, sentiment	learns meaning from sequences
Audio	speech-to-text, music	learns from raw waveforms/spectrograms
Complex signals	sensor fusion, video	patterns too rich to hand-code

The common thread: **you can't easily write the features**. Nobody can enumerate "what makes this pixel-grid a cat," so you let the network learn it.

1.2 What deep learning is *not* the answer for

WHEN NOT TO USE DEEP LEARNING

For **tabular data** — rows of numbers and categories, the typical business dataset — **gradient-boosted trees (XGBoost/LightGBM) usually beat neural nets** and train in seconds. For **small datasets** (hundreds to low thousands of rows) deep nets overfit and underperform simpler models. When you need **interpretability** (why did it decide that?), a linear model or a tree is far easier to explain. And when a **rule or a classic ML model already hits the bar**, the extra complexity, compute, and data hunger of DL buys you nothing. Reach for deep learning for perceptual data and large scale — not by default.

ENGINEER'S MENTAL MODEL

Deep learning trades **engineering effort for data and compute**. Classic ML asks you to hand-craft features and rewards you with fast, cheap, interpretable models. Deep learning skips feature engineering — the network learns features — but demands lots of labelled data, a GPU, longer training, and gives up interpretability. Pick the side of that trade your problem actually needs.

1.3 The data and compute reality

Deep models are hungry. A from-scratch image classifier wants tens of thousands of labelled examples; a from-scratch language model wants far more. Two things make DL practical for mortals anyway, and this book leans on both:

- **Transfer learning** (Part 9) — start from a model pretrained on huge data and fine-tune on your small dataset. This is how most real DL projects work.
- **GPUs** — training is massively parallel matrix math; a GPU is 10-100× faster than CPU (Part 10). Free ones (Colab, Kaggle) get you started.

COMMON MISTAKE

Reaching for a deep neural network on a 2,000-row spreadsheet because it sounds state-of-the-art, then losing to a five-minute XGBoost baseline. Always train the **simple baseline first** (logistic regression, a gradient-boosted tree). If deep learning can't beat it — and on small tabular data it usually can't — you've saved yourself enormous complexity. The baseline is not optional.

EXERCISE 1.1

For three problems (one image, one text, one tabular), decide honestly whether deep learning is warranted using the criteria above: is the input perceptual? is there enough data? do you need interpretability? would boosting or a rule suffice? Predict which will and won't benefit from DL — you'll likely find only the perceptual ones clearly justify it.

Part 2 · Tensors — the Data Structure

Everything in PyTorch is a **tensor**: an n-dimensional array, like a NumPy array but with two superpowers — it can live on a **GPU**, and it can **track gradients** (Part 3). Master tensors and the rest of PyTorch is assembly.

2.1 Creating and inspecting

PYTHON

```
import torch

x = torch.tensor([[1., 2.], [3., 4.]]) # from data
z = torch.zeros(3, 4)                  # shape (3,4) of zeros
r = torch.randn(2, 3)                  # random normal
x.shape                               # torch.Size([2, 2]) — ALWAYS know your shapes
x.dtype                               # torch.float32
x.device                              # cpu (or cuda:0)
```

If you know NumPy, you know 80% of this — `torch.from_numpy(arr)` and `tensor.numpy()` convert both ways (sharing memory on CPU).

2.2 Shapes are the whole game

Most PyTorch bugs are **shape mismatches**. Deep learning is a pipeline of matrix operations, and every layer expects a specific input shape and produces another. The conventions you'll see constantly:

DIAGRAM

```
images: (batch, channels, height, width)    e.g. (32, 3, 224, 224)
sequences: (batch, length, features)
tabular: (batch, features)
```

The **batch dimension comes first** — you always process many examples at once (Part 6). "batch of 32 RGB 224×224 images" is `(32, 3, 224, 224)`.

ENGINEER'S MENTAL MODEL

Treat shapes like types. Before every layer, ask "what shape goes in, what shape comes out?" and `print(x.shape)` liberally while building — it's your `console.log`. A huge fraction of "it won't run" errors are a `(batch, features)` where a `(batch, channels, h, w)` was expected. Shape-checking is the debugging skill of deep learning.

2.3 Operations, broadcasting, and reshaping

PYTHON

```
a + b; a * b; a @ b           # elementwise add/mul, matrix multiply (@)
x.sum(dim=0); x.mean(dim=1)   # reduce along an axis
x.reshape(4, 1); x.view(-1)   # change shape (-1 = "infer this dimension")
x.unsqueeze(0)                # add a dimension (e.g. make a batch of 1)
x.permute(0, 3, 1, 2)         # reorder dims (e.g. HWC -> CHW)
```

Broadcasting (like NumPy) lets differently-shaped tensors combine by auto-expanding compatible dimensions — powerful and a common source of silent bugs when shapes line up by accident.

2.4 GPU tensors

PYTHON

```
device = "cuda" if torch.cuda.is_available() else "cpu"
x = x.to(device)                # move a tensor to the GPU
# a golden rule: model and data must be on the SAME device
```

COMMON MISTAKE

Mixing devices — model on GPU, a batch still on CPU — which throws "Expected all tensors to be on the same device." Move **both** the model and each batch to the same `device` (Part 10 shows the pattern). The mirror-image mistake is calling `.numpy()` on a GPU tensor that still needs a gradient — detach and move to CPU first (`x.detach().cpu().numpy()`).

PRODUCTION TIP

Keep a `device` variable at the top of every script and route the model and every batch through `.to(device)`. Written once, the same code runs on CPU for debugging and GPU for training with no changes — the standard portable pattern.

EXERCISE 2.1

Create tensors of shapes `(32, 3, 28, 28)` and `(28, 28)`. Practise: matrix multiply two compatible tensors, reduce one along a chosen axis, reshape a `(32, 3, 28, 28)` image batch to `(32, 2352)` for a linear layer, and add a batch dimension to a single image with `unsqueeze`. Print `.shape` after each step until predicting the output shape becomes automatic.

Part 3 · Autograd — How Learning Actually Happens

Training a network means adjusting weights to reduce a loss, and to adjust a weight you need its **gradient** — how much the loss changes if you nudge that weight. PyTorch computes all those gradients automatically with **autograd**. Understanding it demystifies the whole training loop.

3.1 Tensors that track history

Set `requires_grad=True` and PyTorch records every operation into a **computational graph**, so it can later run them backward to get gradients:

PYTHON

```
import torch
x = torch.tensor(3.0, requires_grad=True)
y = x**2 + 2*x + 1          # PyTorch builds a graph as this runs
y.backward()                # walk the graph backward, compute dy/dx
print(x.grad)               # tensor(8.) - dy/dx = 2x+2 = 8 at x=3
```

`backward()` fills in `.grad` for every tensor that required one. That's the whole mechanism: build a graph on the forward pass, differentiate it on the backward pass.

ENGINEER'S MENTAL MODEL

Autograd is automatic differentiation, not numeric guessing: it knows the derivative rule for each operation and applies the chain rule along the graph you built. You get exact gradients for free, however complex the model. This is the single feature that makes deep learning practical — nobody hand-derives gradients for a million-parameter network.

3.2 The four-step dance (the heart of every training loop)

Every parameter update is these four calls, in this order:

PYTHON

```
pred = model(x)              # 1. forward: compute prediction (builds graph)
loss = loss_fn(pred, y)      # 2. loss: how wrong are we?
optimizer.zero_grad()        # 3a. reset gradients to zero
loss.backward()              # 3b. backward: compute every gradient
optimizer.step()             # 4. update: nudge weights down the gradient
```

Miss any step and training silently breaks. The most infamous omission is `zero_grad()`.

3.3 Why `zero_grad()` matters

PyTorch **accumulates** gradients — each `backward()` *adds* to `.grad` rather than replacing it. That's occasionally useful, but normally you want each step's gradient fresh, so you zero them first. Forget it and gradients from all previous batches pile up, and training goes haywire.

COMMON MISTAKE

Forgetting `optimizer.zero_grad()`, so gradients accumulate across batches and the model updates on a growing sum of stale gradients — training diverges or behaves bizarrely, with no error message. It's the #1 hand-written-loop bug. Burn the four-step order into memory: **zero** → **forward/loss** → **backward** → **step** (zeroing can go first or just before backward, as long as it's every iteration).

3.4 Turning autograd off for inference

Tracking the graph costs memory and time, and you don't need gradients when just *using* a trained model. Wrap inference in `torch.no_grad()`:

PYTHON

```
model.eval()                # switch layers like dropout/batchnorm to eval mode
with torch.no_grad():       # don't build the graph – faster, less memory
    preds = model(x_test)
```

PRODUCTION TIP

`model.eval()` and `torch.no_grad()` are **different** and you need both for inference: `eval()` changes the *behaviour* of dropout and batch-norm layers (Part 8), while `no_grad()` stops gradient tracking for speed/memory. Forgetting `eval()` is a classic bug — your model gives subtly wrong, randomised predictions at test time because dropout is still active.

EXERCISE 3.1

By hand, minimise $f(w) = (w - 5)^2$ with autograd: start `w` at 0 with `requires_grad=True`, and loop — compute `f`, `backward()`, then (inside `torch.no_grad()`) `w -= 0.1 * w.grad`, then `w.grad.zero_()`. Watch `w` march toward 5. You've just implemented gradient descent by hand — exactly what the optimizer automates.

Part 4 · Building Networks with nn.Module

You rarely wire raw tensor math by hand. PyTorch's `nn` package gives you **layers** as building blocks and `nn.Module` as the container that holds them, tracks their weights, and defines the forward pass. This is how every real model is structured.

4.1 The layers you'll use constantly

Layer	What it does	Used for
<code>nn.Linear(in, out)</code>	fully-connected (matrix multiply + bias)	tabular, heads
<code>nn.Conv2d(...)</code>	convolution	images (Part 7)
<code>nn.ReLU()</code>	activation (adds non-linearity)	almost everywhere
<code>nn.Dropout(p)</code>	randomly zero units (regularization)	reduce overfitting
<code>nn.BatchNorm...</code>	normalize activations	stabilize training
<code>nn.Embedding(...)</code>	lookup table for tokens/categories	text, categoricals

Activations matter: without a non-linear activation (ReLU and friends) between linear layers, stacking layers collapses to a single linear layer — the network can't learn anything a straight line couldn't.

4.2 Defining a model

PYTHON

```
import torch.nn as nn

class Net(nn.Module):
    def __init__(self, in_features, n_classes):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, 128), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(128, 64),          nn.ReLU(),
            nn.Linear(64, n_classes),    # raw scores ("logits")
        )
    def forward(self, x):
        return self.net(x)

model = Net(in_features=20, n_classes=3)
```

ENGINEER'S MENTAL MODEL

`nn.Module` is a class that owns its parameters. Define layers in `__init__` (PyTorch auto-registers their weights so the optimizer can find them via `model.parameters()`), and describe the data flow in `forward`. You call `model(x)` — never `model.forward(x)` directly — so PyTorch's hooks run. `nn.Sequential` is sugar for "run these in order"; write a custom `forward` when the flow branches.

4.3 Loss functions and what the last layer outputs

The output layer and loss must match the task — and a classic trap lives here:

PYTHON

```
loss_fn = nn.CrossEntropyLoss()    # multi-class classification
loss_fn = nn.BCEWithLogitsLoss()    # binary classification
loss_fn = nn.MSELoss()              # regression
```

For classification, output **raw logits** (no softmax/sigmoid in the model) — `CrossEntropyLoss` and `BCEWithLogitsLoss` apply it internally and more stably.

COMMON MISTAKE

Adding a `softmax` to the end of the model *and* using `CrossEntropyLoss`, which applies softmax again — so you softmax twice, the gradients weaken, and the model trains poorly. Output raw logits from the model; let the loss handle the softmax/sigmoid. (For *inference*, apply softmax/sigmoid yourself to turn logits into probabilities.)

PRODUCTION TIP

Sanity-check a new model with **one forward pass before training anything**:

`model(torch.randn(2, in_features)).shape` should equal `(2, n_classes)`. This catches shape and wiring bugs in a second, before you waste a training run discovering the output layer is the wrong size.

EXERCISE 4.1

Build an `nn.Module` MLP for a 20-feature, 3-class problem: two hidden layers with ReLU and a dropout, outputting 3 logits. Run one forward pass on a random `(8, 20)` batch and confirm the output is `(8, 3)`. Count the parameters with `sum(p.numel() for p in model.parameters())`. Then remove the ReLUs and reason about why the network is now no more expressive than a single linear layer.

Part 5 · The Training Loop

This is the chapter everything led to: the loop that actually trains a model. In PyTorch you write it yourself (unlike Keras' `.fit()`), which is a feature — you see every step and can debug it. Learn this loop once and you can train anything.

5.1 The canonical loop

PYTHON

```
import torch
device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(n_epochs):
    model.train()                                # training mode (dropout/batchnorm active)
    for xb, yb in train_loader:                  # batches (Part 6)
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()                    # 1. clear old gradients
        pred = model(xb)                         # 2. forward
        loss = loss_fn(pred, yb)                 # 3. loss
        loss.backward()                          # 4. gradients
        optimizer.step()                         # 5. update weights

    validate(model, val_loader)                  # check held-out performance each epoch
```

An **epoch** is one full pass over the training data; a **batch** is one group of examples processed together (Part 6). That inner five-line body is the four-step autograd dance from Part 3 — this is where it lives.

ENGINEER'S MENTAL MODEL

PyTorch hands you the loop instead of hiding it behind `.fit()`, so training is just a Python `for` loop you can `print` inside, set a breakpoint in, and modify. Everything fancy — schedulers, mixed precision, gradient clipping — is a line added to *this* loop. Own the loop and nothing about training is a black box.

5.2 Optimizers and learning rate

The **optimizer** decides how to turn gradients into weight updates; **Adam** is the reliable default. The **learning rate** (`lr`) is the most important hyperparameter — the step size:

- Too high → loss explodes to `NaN` or bounces and never settles.

- Too low → trains agonisingly slowly, or gets stuck.
- `1e-3` is a good Adam starting point; tune from there.

5.3 Always validate on held-out data

Track loss on a **validation set** the model doesn't train on. It's your only honest signal of generalisation and the basis for spotting overfitting (Part 8):

PYTHON

```
model.eval()
with torch.no_grad():
    val_loss = sum(loss_fn(model(xb.to(device)), yb.to(device)) for xb, yb in
    val_loader)
```

5.4 Read the two loss curves together

Plot training and validation loss per epoch. **Both falling** → learning. **Training falls, validation rises** → overfitting (Part 8). **Neither falls** → too-low learning rate, a bug, or too-weak a model. The curves diagnose almost everything.

COMMON MISTAKE

Loss going to `NaN` a few steps in — almost always the **learning rate is too high** (or unnormalised inputs, or a numerically unstable manual softmax). When you see `NaN`, lower the LR by 10× first; it fixes it most of the time. The opposite symptom, a totally flat loss, usually means the LR is too low, the data isn't shuffled, or `zero_grad()` is missing.

PRODUCTION TIP

Before a long run, **overfit a single batch on purpose**: train on ~8 examples and confirm the model can drive their loss to near zero. If it can't, the model or loss is broken and no amount of data or epochs will help — you've found the bug in 20 seconds instead of after an hour of training. It's the fastest deep-learning sanity check there is.

EXERCISE 5.1

Train the MLP from Part 4 on a real dataset (e.g. scikit-learn's digits or a tabular set): write the full loop, track train and validation loss per epoch, and plot both. First do the single-batch overfit test. Then deliberately set `lr=10` and watch it diverge to `NaN`, and `lr=1e-6` and watch it crawl — feel the learning rate's effect directly.

Part 6 · Feeding Data: Dataset & DataLoader

Models train on **batches**, not one example at a time and not the whole dataset at once. PyTorch's `Dataset` and `DataLoader` handle batching, shuffling, and parallel loading so your GPU never sits idle waiting for data. This is the plumbing that keeps training fast.

6.1 Dataset — one example at a time

A `Dataset` just answers "how many examples?" and "give me example `i`":

PYTHON

```
from torch.utils.data import Dataset
import torch

class CSVDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.long)
    def __len__(self): return len(self.X)
    def __getitem__(self, i): return self.X[i], self.y[i]
```

For common cases you rarely write one — `torchvision.datasets.ImageFolder` reads a folder of class-named subfolders, and there are built-in datasets for learning.

6.2 DataLoader — batching, shuffling, parallelism

PYTHON

```
from torch.utils.data import DataLoader

train_loader = DataLoader(train_ds, batch_size=64, shuffle=True, num_workers=4)
val_loader = DataLoader(val_ds, batch_size=64, shuffle=False)
for xb, yb in train_loader:    # xb: (64, ...features), yb: (64,)
```

- `batch_size` — examples per step (32–256 typical). Bigger = smoother gradients + more memory.
- `shuffle=True` **for training** — so the model doesn't learn the data order.
- `num_workers` — parallel loading processes so data prep overlaps GPU compute.

ENGINEER'S MENTAL MODEL

`DataLoader` is a producer feeding batches to the training loop's consumer — conceptually the same as a queue of work items with parallel workers (your Queues cookbook). `num_workers` is worker parallelism: it prepares the next batches while the GPU chews the current one, so an expensive dataset doesn't starve the GPU. Shuffle training data; never shuffle validation.

6.3 Why batches at all

- **Memory** — the whole dataset rarely fits on the GPU; batches do.
- **Speed** — GPUs are built to process a batch in parallel; one example wastes them.
- **Better learning** — the slight noise from batch-to-batch gradients actually helps generalisation (this is the "stochastic" in stochastic gradient descent).

6.4 Normalize your inputs

Neural nets train far better on inputs scaled to a small, consistent range (roughly mean 0, std 1, or pixels in [0,1]). Fit the scaling on the **training set** and apply the same transform to validation/test.

COMMON MISTAKE

Two data traps that quietly wreck results: (1) **not normalising inputs**, so wildly-scaled features make training unstable or glacial; and (2) **data leakage** — fitting your scaler (or any preprocessing) on the full dataset *before* the train/test split, so test statistics bleed into training and your reported accuracy is a lie. Always split first, then fit preprocessing on train only.

PRODUCTION TIP

If GPU utilisation is low and training feels slow, the bottleneck is usually **data loading, not the model** — the GPU is starving. Raise `num_workers`, use a faster storage/format, precompute expensive transforms, and set `pin_memory=True`. A fast model fed by a slow `DataLoader` runs at the speed of the `DataLoader`.

EXERCISE 6.1

Wrap a dataset in a custom `Dataset` and a `DataLoader` with `batch_size=64`, `shuffle=True`. Iterate one epoch and confirm the batch shapes. Add input normalisation fit on the training split only. Then deliberately fit the scaler on the full dataset before splitting, retrain, and observe the (falsely) improved validation score — leakage in action.

Part 7 · Convolutional Networks for Images

Images are where deep learning first crushed classic ML, and the **convolutional neural network (CNN)** is why. A CNN learns visual features — edges, textures, shapes — directly from pixels, with far fewer parameters than a fully-connected net. This chapter builds one and explains why it works.

7.1 Why not just a big Linear layer?

Flattening a $224 \times 224 \times 3$ image into a vector and feeding a `nn.Linear` throws away the 2D structure and needs a colossal number of weights. A **convolution** instead slides a small learnable filter across the image, reusing the same weights everywhere. That gives two things for free:

- **Parameter sharing** — one small filter covers the whole image (few weights).
- **Translation invariance** — a cat is a cat wherever it appears, because the same filter looks everywhere.

ENGINEER'S MENTAL MODEL

A convolution filter is a small pattern detector scanned across the image, firing wherever its pattern appears — like running the same regex over every position of a string instead of a separate rule per position. Early layers learn simple patterns (edges); deeper layers compose them into complex ones (eyes, wheels). The network builds its own feature hierarchy.

7.2 The building blocks

PYTHON

```
import torch.nn as nn

cnn = nn.Sequential(
    nn.Conv2d(3, 32, kernel_size=3, padding=1), nn.ReLU(), # (B,3,H,W) -> (B,32,H,W)
    nn.MaxPool2d(2), # halve H,W
    nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Flatten(),
    nn.Linear(64 * 8 * 8, 128), nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(128, n_classes),
)
```

- **Conv2d** — the learnable filters; `out_channels` = how many features.
- **MaxPool2d** — downsample, keeping the strongest signal (shrinks $H \times W$).

- **Flatten + Linear** — the classifier "head" that turns features into class scores.

Channels grow deeper (3→32→64) as spatial size shrinks (pool by pool) — the network trades resolution for richer features.

7.3 Getting the flattened size right

The `nn.Linear` after `Flatten` needs the exact number of features coming out of the conv stack (`channels × height × width`). Miscalculation and it errors immediately.

COMMON MISTAKE

Hard-coding the wrong `Linear` input size after the conv layers — the #1 CNN bug. Rather than compute `64*8*8` by hand (and get the pooling arithmetic wrong), run a dummy tensor through the conv part and read the shape: `conv(torch.randn(1,3,32,32)).shape`. Use that number, or use `nn.LazyLinear` / `nn.AdaptiveAvgPool2d` to sidestep the arithmetic entirely.

7.4 In practice, you won't build from scratch

A from-scratch CNN is great for learning, but for real image tasks you'll almost always **start from a pretrained model** (ResNet, EfficientNet) and fine-tune it — that's Part 9, and it's how you get strong results on modest data. Build one CNN by hand to understand it, then stand on giants.

PRODUCTION TIP

Use **data augmentation** for images — random flips, crops, rotations, colour jitter (`torchvision.transforms`) applied during training. It multiplies your effective dataset and is one of the highest-leverage ways to cut overfitting on image models (Part 8), often worth more than a bigger network. Augment training only, never validation.

EXERCISE 7.1

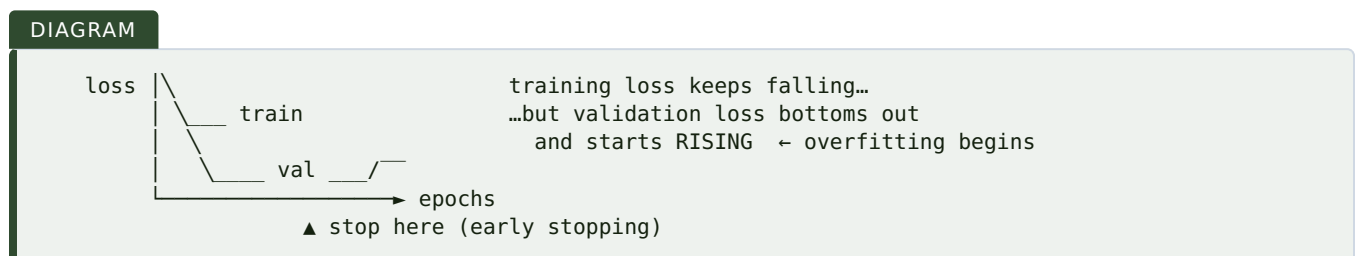
Build and train a small CNN on CIFAR-10 or Fashion-MNIST. First determine the post-conv flattened size with a dummy forward pass. Train it, plot train/val accuracy, then add `torchvision` augmentation (random crop + flip) and measure the improvement in validation accuracy. Compare against a plain MLP on the same images to feel why convolution wins.

Part 8 · Overfitting & Regularization

Deep networks have so many parameters they can **memorise** the training set — scoring perfectly on it while failing on anything new. The whole art of making a model *generalise* is regularization: techniques that stop memorisation. This is often the difference between a demo and a useful model.

8.1 Spotting overfitting

The signature is the gap between the two loss curves (Part 5):



Training loss down, validation loss up = the model is memorising. Everything below narrows that gap.

8.2 The regularization toolkit

- **More data** — the best cure; augmentation (Part 7) is cheap synthetic data.
- **Early stopping** — stop when validation loss stops improving; keep the best checkpoint. Simple and very effective.
- **Dropout** — randomly zero a fraction of units during training so the net can't rely on any one; `nn.Dropout(0.2-0.5)`.
- **Weight decay** — penalise large weights (L2), via the optimizer's `weight_decay=`; pushes toward simpler functions.
- **Batch normalization** — normalize activations between layers; stabilises and mildly regularises.
- **A smaller model** — fewer parameters can't memorise as much; don't reach for a giant network on a small dataset.

PYTHON

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
# nn.Dropout(0.3) between layers; nn.BatchNorm2d(channels) after convs
```

ENGINEER'S MENTAL MODEL

Overfitting is memorising the test answers instead of learning the subject. Regularization forces the model to find patterns that generalise by making memorisation harder — dropout randomly hides its notes, weight decay penalises over-complex rules, early stopping ends the exam before cramming sets in. You're deliberately handicapping training-set performance to buy real-world performance.

8.3 eval vs train mode really matters here

Dropout and batch-norm behave **differently** in training vs inference. You must call `model.train()` during the training loop and `model.eval()` before validation/inference — or dropout stays on at test time and batch-norm uses the wrong statistics, giving wrong, noisy predictions.

COMMON MISTAKE

Forgetting `model.eval()` at inference, so dropout is still randomly zeroing units and batch-norm is still updating from the eval batch — your predictions become noisy and worse, and it looks like the model "got worse after training." Pair every `model.train()` in the loop with a `model.eval()` before you measure or predict.

WHEN NOT TO ADD MORE REGULARIZATION

If the model is **underfitting** — high loss on *both* train and validation — it isn't memorising; it's too weak or under-trained. Piling on dropout and weight decay makes it worse. First get the model to overfit a bit (proof it *can* learn), *then* regularise it back. Diagnose which problem you have before reaching for the cure.

EXERCISE 8.1

Train a deliberately oversized model on a small dataset until validation loss clearly rises while training loss keeps falling — see overfitting happen. Then add, one at a time, dropout, weight decay, and early stopping, and watch the validation curve improve. Confirm you're calling `model.train()` and `model.eval()` in the right places by toggling them and observing the effect.

Part 9 · Transfer Learning — the Technique You'll Actually Use

Almost nobody trains a large network from scratch. Instead you take a model **pretrained** on a huge dataset and adapt it to your (much smaller) problem — **transfer learning**. It's how you get state-of-the-art results with hundreds of examples and an afternoon instead of millions of examples and a data centre. For most real DL work, this is the technique.

9.1 Why it works

A model trained on millions of images has already learned general visual features — edges, textures, shapes — in its early layers. Those features transfer to almost any image task; only the final classification layer is specific to the original 1000 classes. So you **keep the learned features and replace the head**.

DIAGRAM

```
[pretrained backbone: general features] → [new head: your classes]
keep (already knows how to "see")         train (learns your task)
```

ENGINEER'S MENTAL MODEL

Transfer learning is depending on a well-tested library instead of writing everything yourself. The backbone is a battle-tested feature extractor someone else trained at great expense; you import it and write only the thin application-specific layer on top. Training from scratch is reimplementing the standard library — occasionally necessary, usually wasteful.

9.2 The recipe (image example)

PYTHON

```
import torch, torch.nn as nn
from torchvision import models

model = models.resnet18(weights="IMAGENET1K_V1") # pretrained backbone
for p in model.parameters():                     # 1. freeze the backbone
    p.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, n_classes) # 2. new head (trainable)
# 3. train — only the new head's weights update
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

Two phases in practice:

1. **Feature extraction** — freeze the backbone, train only the new head. Fast, works with little data.
2. **Fine-tuning** — then optionally unfreeze the top backbone layers and train with a **very low** learning rate to adapt them slightly to your data.

9.3 It's not just images

The same idea dominates NLP: you take a pretrained language model from the Hugging Face `transformers` hub and fine-tune it on your text task. Pretrained backbones exist for vision, text, audio, and more — starting from one is the default move, not an optimisation.

PRODUCTION TIP

Match the preprocessing to the pretrained model exactly — the same input size, normalisation mean/std, and channel order it was trained with (the model card / weights transform specifies them). Feed it differently-normalised images and the transferred features misfire, quietly tanking accuracy. This mismatch is the most common reason "transfer learning didn't work for me."

COMMON MISTAKE

Fine-tuning the whole pretrained network at a **high** learning rate. Big updates erase (catastrophically forget) the valuable pretrained features you came for, and on small data you overfit instantly. Start by training only the new head; unfreeze gradually and use a much smaller LR (e.g. $1e-5$) when you do. Gentle is the whole point.

WHEN NOT TO FINE-TUNE THE BACKBONE

With a **small** dataset, don't unfreeze the backbone at all — freeze it, train just the head, and stop. There isn't enough data to improve millions of backbone weights without overfitting, and you risk destroying good features for no gain. Full fine-tuning pays off mainly when your dataset is large and somewhat different from the backbone's original domain.

EXERCISE 9.1

Take a small image dataset (a few hundred images, 3–5 classes). Load a pretrained ResNet, freeze it, replace the head, and train only the head — note how quickly it reaches good accuracy versus a from-scratch CNN (Part 7) on the same data. Then unfreeze the top block and fine-tune at `lr=1e-5`; measure whether it helps or overfits, given your dataset size.

Part 10 · GPUs, Saving & Serving

A trained model that lives only in a notebook helps nobody. This chapter covers the practical last mile: training on a GPU, saving and loading correctly, and serving the model behind an API — where your backend skills take over.

10.1 GPU training, portably

PYTHON

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)
# in the loop:
xb, yb = xb.to(device), yb.to(device)
```

That's the whole GPU story for one card: move the model and every batch to `device`. Written this way the same script runs on CPU (debugging) and GPU (training). Multi-GPU and mixed precision exist for scale, but single-GPU covers almost everything you'll do.

ENGINEER'S MENTAL MODEL

A GPU is a massively parallel matrix-multiply engine, and a neural net is mostly matrix multiplies — hence 10-100× speedups. You don't write GPU code; you just move tensors onto it and PyTorch runs the ops there. Free GPUs (Colab, Kaggle) are plenty for learning and small projects; rent cloud GPUs when you outgrow them.

10.2 Saving and loading — save the state_dict

Save the **weights** (`state_dict`), not the whole Python object — it's the portable, recommended way:

PYTHON

```
torch.save(model.state_dict(), "model.pt")           # save weights

# to load: recreate the architecture, then load weights into it
model = Net(in_features=20, n_classes=3)
model.load_state_dict(torch.load("model.pt", map_location="cpu"))
model.eval()                                         # ALWAYS before inference
```

COMMON MISTAKE

Two saving traps: (1) `torch.save(model)` (the whole object) instead of `model.state_dict()` — it pickles file paths and class references and breaks when your code moves; save the `state_dict` and recreate the architecture to load. (2) Forgetting `model.eval()` after loading, so dropout/batch-norm run in training mode and predictions come out wrong and noisy (Parts 3, 8). Both are silent — no error, just bad results.

10.3 Serving behind an API

Inference is a forward pass; wrap it in FastAPI (your Python book) like any model:

PYTHON

```
from fastapi import FastAPI
import torch

app = FastAPI()
model = Net(...); model.load_state_dict(torch.load("model.pt", map_location="cpu"));
model.eval()

@app.post("/predict")
def predict(features: list[float]):
    x = torch.tensor([features], dtype=torch.float32)
    with torch.no_grad():                                # no gradients at inference
        logits = model(x)
        probs = torch.softmax(logits, dim=1)              # logits -> probabilities here
    return {"probs": probs[0].tolist()}
```

10.4 Beyond a plain endpoint

- **ONNX / TorchScript** — export the model to a framework-independent format to run it outside Python (mobile, C++, other runtimes) and often faster.
- **Batching requests** — group incoming requests to use the GPU efficiently under load (a serving framework like TorchServe or Triton does this for you).
- **Versioning & monitoring** — track which model is deployed and watch for input drift, exactly as in the ML book's MLOps chapter.

PRODUCTION TIP

For CPU-only serving of a small model, export to **ONNX** and run it with ONNX Runtime — it's typically faster than PyTorch on CPU, has a smaller footprint, and drops the heavy training dependency from your production image. Keep the full PyTorch stack for *training*; ship the lean runtime for *inference*.

WHEN NOT TO SERVE IT YOURSELF

If your model is a standard pretrained one (a common vision or language model), a hosted inference API or an off-the-shelf serving runtime may be cheaper and more reliable than hand-rolling and maintaining your own GPU service. Build custom serving when you have a custom model, real scale, or latency/privacy constraints — not by default.

EXERCISE 10.1

Take a model you trained earlier: save its `state_dict`, reload it into a fresh architecture instance with `map_location="cpu"`, call `eval()`, and confirm predictions match the original. Then serve it behind a FastAPI `/predict` endpoint that returns probabilities, using `torch.no_grad()`. Bonus: export to ONNX and compare CPU inference speed against PyTorch.

Part 11 · Capstone & Cheat Sheet

11.1 Capstone — train and ship an image classifier

Put it all together on a real image task:

1. **Justify it** — confirm deep learning is warranted (perceptual data, enough examples), and note the simple baseline you'd compare against (Part 1).
2. **Data** — a `Dataset` + `DataLoader`, normalised to the pretrained model's spec, with training augmentation (Parts 6, 7, 9).
3. **Model** — start from a **pretrained** backbone, freeze it, replace the head (Part 9). (Build one CNN from scratch once, for understanding — Part 7.)
4. **Train** — the standard loop, Adam, tracking train/val curves; do the single-batch overfit test first (Part 5).
5. **Regularize** — dropout, weight decay, early stopping; watch the val curve and mind `train() / eval()` (Part 8).
6. **Ship** — save the `state_dict`, reload, `eval()`, serve behind FastAPI with `no_grad()`; optionally export to ONNX (Part 10).

A good target: fine-tune a ResNet to classify a small custom image set to strong accuracy, then serve it. That's a real, shippable deep-learning project.

11.2 PyTorch cheat sheet

DIAGRAM

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device); xb.to(device)          model & data on the SAME device

# the training step (memorize this order)
optimizer.zero_grad()                    clear old gradients
pred = model(xb)                          forward
loss = loss_fn(pred, yb)                  loss (logits in, no softmax in model)
loss.backward()                           gradients (autograd)
optimizer.step()                          update weights

model.train() / model.eval()              switch dropout & batchnorm modes
with torch.no_grad(): ...                  inference: no graph, faster
torch.save(model.state_dict(), "m.pt")    save WEIGHTS, not the object
```

11.3 Debugging cheat sheet

DIAGRAM

```
loss = NaN          → learning rate too high (+10); or unnormalised inputs
loss flat           → LR too low, data not shuffled, or missing zero_grad()
shape error         → print(x.shape) before each layer; batch dim is first
val loss rising     → overfitting → dropout / weight decay / early stopping
train & val both high → underfitting → bigger model / train longer / higher LR
```

```
wrong at inference      → forgot model.eval() (dropout/batchnorm still training)
"different device"     → move model AND batch to the same device
can't overfit 1 batch → the model or loss is broken; fix before scaling up
```

11.4 The five rules

1. **Not everything needs deep learning.** Tabular → boosting; small data → simpler models. Baseline first.
2. **Own the training loop.** zero_grad → forward → loss → backward → step, every iteration.
3. **Watch two loss curves.** They diagnose overfitting, underfitting, and bad learning rates.
4. **Transfer learning is the default.** Start from a pretrained backbone; match its preprocessing.
5. **Ship the state_dict, in eval mode, with no_grad.** Save weights, recreate the model, and remember `eval()`.

YOU'RE READY

You can now build networks from tensors and autograd, write and debug the training loop, feed data efficiently, train CNNs, fight overfitting, stand on pretrained models with transfer learning, and serve the result — and, crucially, judge when a simpler model was the smarter call. That judgement plus a working training loop is real deep-learning competence. Go train something that sees.